

C++中的代码重用 - 完整自学手册

目录

1. 基础概念
 2. has-a关系与包含
 3. 私有继承
 4. 多重继承
 5. 类模板
 6. 实践练习
 7. 总结
-

1. 基础概念

1.1 代码重用的重要性

代码重用是C++的核心设计目标之一。通过重用已经测试过的代码，我们可以：

- 提高开发效率
- 减少错误
- 保持代码一致性
- 简化维护工作

1.2 代码重用的主要方式

C++提供了多种代码重用机制：

1. **公有继承** - 建立is-a关系
 2. **包含（组合）** - 建立has-a关系
 3. **私有/保护继承** - 建立has-a关系
 4. **多重继承** - 从多个基类继承
 5. **模板** - 创建通用代码
-

2. has-a关系与包含

2.1 什么是has-a关系

has-a关系表示“拥有”关系，例如：

- 学生**有**姓名
- 汽车**有**引擎

- 房屋有厨房

2.2 包含的实现

包含是通过将其他类的对象作为当前类的成员来实现的。

示例：Student类

CPP

```
#include <string>
#include <valarray>

class Student {
private:
    std::string name;           // 包含string对象
    std::valarray<double> scores; // 包含valarray对象

public:
    // 构造函数
    Student() : name("Null Student"), scores() {}

    explicit Student(const std::string& s)
        : name(s), scores() {}

    Student(const std::string& s, int n)
        : name(s), scores(n) {}

    // 成员函数
    double Average() const;
    const std::string& Name() const;
    double& operator[](int i);
    double operator[](int i) const;
};
```

2.3 valarray类简介

valarray是C++标准库提供的数值数组模板类：

cpp

```
#include <valarray>
```

```
// 基本用法
```

```
std::valarray<double> v1;           // 空数组  
std::valarray<int> v2(8);           // 8个int元素  
std::valarray<int> v3(10, 8);       // 8个元素，都初始化为10  
std::valarray<int> v4 = {20, 32, 17, 9}; // C++11初始化列表
```

```
// 常用方法
```

```
v1.size();    // 返回元素个数  
v1.sum();     // 返回所有元素的总和  
v1.max();     // 返回最大元素  
v1.min();     // 返回最小元素
```

2.4 完整的Student类实现

```
cpp
```

```

// studentc.h
#ifndef STUDENTC_H
#define STUDENTC_H

#include <iostream>
#include <string>
#include <valarray>

class Student {
private:
    typedef std::valarray<double> ArrayDb;
    std::string name;
    ArrayDb scores;

    // 私有辅助方法
    std::ostream& arr_out(std::ostream& os) const;

public:
    Student() : name("Null Student"), scores() {}

    explicit Student(const std::string& s)
        : name(s), scores() {}

    explicit Student(int n)
        : name("Nully"), scores(n) {}

    Student(const std::string& s, int n)
        : name(s), scores(n) {}

    Student(const std::string& s, const ArrayDb& a)
        : name(s), scores(a) {}

    Student(const char* str, const double* pd, int n)
        : name(str), scores(pd, n) {}

    ~Student() {}

    double Average() const;
    const std::string& Name() const;
    double& operator[](int i);
    double operator[](int i) const;

    // 友元函数
    friend std::istream& operator>>(std::istream& is, Student& stu);
    friend std::istream& getline(std::istream& is, Student& stu);
    friend std::ostream& operator<<(std::ostream& os, const Student& stu);

```

```
};
```

```
#endif
```

CPP

```
// studentc.cpp
#include "studentc.h"

double Student::Average() const {
    if (scores.size() > 0)
        return scores.sum() / scores.size();
    else
        return 0;
}

const std::string& Student::Name() const {
    return name;
}

double& Student::operator[](int i) {
    return scores[i];
}

double Student::operator[](int i) const {
    return scores[i];
}

std::ostream& Student::arr_out(std::ostream& os) const {
    int i;
    int lim = scores.size();
    if (lim > 0) {
        for (i = 0; i < lim; i++) {
            os << scores[i] << " ";
            if (i % 5 == 4)
                os << std::endl;
        }
        if (i % 5 != 0)
            os << std::endl;
    } else {
        os << " empty array";
    }
    return os;
}

std::istream& operator>>(std::istream& is, Student& stu) {
    is >> stu.name;
    return is;
}

std::istream& getline(std::istream& is, Student& stu) {
    getline(is, stu.name);
}
```

```
        return is;
    }

    std::ostream& operator<<(std::ostream& os, const Student& stu) {
        os << "Scores for " << stu.name << ":\n";
        stu.arr_out(os);
        return os;
    }
```

2.5 关键点

- 1. **初始化顺序**：成员对象按在类中声明的顺序初始化，而不是按初始化列表的顺序
- 2. **explicit关键字**：防止单参数构造函数的隐式转换
- 3. **接口访问**：被包含对象的接口不会自动成为包含类的接口

3. 私有继承

3.1 私有继承基本概念

私有继承也可以建立has-a关系，但实现方式不同：

```
CPP

class Student : private std::string, private std::valarray<double> {
    // 类定义
};
```

3.2 私有继承vs包含的区别

特性	包含	私有继承
对象命名	有明确的成员对象名称	无名称的子对象
初始化方式	使用成员名	使用类名
方法调用	通过对象名调用	通过类名和作用域解析
多个同类对象	可以有多个	只能有一个

3.3 私有继承实现示例

```
CPP
```

```

// studenti.h
#ifndef STUDENTI_H
#define STUDENTI_H

#include <iostream>
#include <string>
#include <valarray>

class Student : private std::string, private std::valarray<double> {
private:
    typedef std::valarray<double> ArrayDb;
    std::ostream& arr_out(std::ostream& os) const;

public:
    Student() : std::string("Null Student"), ArrayDb() {}

    explicit Student(const std::string& s)
        : std::string(s), ArrayDb() {}

    explicit Student(int n)
        : std::string("Nully"), ArrayDb(n) {}

    Student(const std::string& s, int n)
        : std::string(s), ArrayDb(n) {}

    Student(const char* str, const double* pd, int n)
        : std::string(str), ArrayDb(pd, n) {}

    ~Student() {}

    double Average() const;
    const std::string& Name() const;
    double& operator[](int i);
    double operator[](int i) const;

    // 友元函数
    friend std::istream& operator>>(std::istream& is, Student& stu);
    friend std::istream& getline(std::istream& is, Student& stu);
    friend std::ostream& operator<<(std::ostream& os, const Student& stu);
};

#endif

```

3.4 访问基类方法和对象


```

// 访问基类方法
double Student::Average() const {
    if (ArrayDb::size() > 0)
        return ArrayDb::sum() / ArrayDb::size();
    else
        return 0;
}

// 访问基类对象 - 使用强制类型转换
const std::string& Student::Name() const {
    return (const std::string&)*this;
}

// 在友元函数中访问基类
std::ostream& operator<<(std::ostream& os, const Student& stu) {
    os << "Scores for " << (const std::string&)stu << ":\n";
    stu.arr_out(os);
    return os;
}

```

3.5 使用声明重新定义访问权限

```

CPP

class Student : private std::string, private std::valarray<double> {
public:
    using std::valarray<double>::min; // 公开min方法
    using std::valarray<double>::max; // 公开max方法
    using std::valarray<double>::operator[]; // 公开[]运算符
    // ...
};

```

3.6 何时使用私有继承

使用包含的情况（推荐）：

- 大多数has-a关系
- 需要多个同类型对象
- 关系相对简单

使用私有继承的情况：

- 需要访问基类的保护成员
- 需要重新定义虚函数
- 需要更抽象的实现关系

4. 多重继承

4.1 多重继承基本概念

多重继承允许一个类从多个基类派生：

CPP

```
class SingingWaiter : public Waiter, public Singer {  
    // 类定义  
};
```

4.2 多重继承的问题

4.2.1 同名方法的二义性

CPP

```
SingingWaiter newhire("Elise Hawks", 2005, 6, soprano);  
// newhire.Show(); // 错误：不知道调用哪个Show()  
  
// 解决方案：使用作用域解析  
newhire.Singer::Show(); // 调用Singer版本  
newhire.Waiter::Show(); // 调用Waiter版本
```

4.2.2 多个基类对象的问题

当从多个派生类继承同一个基类时，会产生多个基类对象副本：

CPP

```
class Worker { /* ... */ };  
class Waiter : public Worker { /* ... */ };  
class Singer : public Worker { /* ... */ };  
class SingingWaiter : public Waiter, public Singer { /* ... */ };  
// SingingWaiter包含两个Worker对象！
```

4.3 虚基类解决方案

CPP

```
class Waiter : virtual public Worker { /* ... */ };  
class Singer : virtual public Worker { /* ... */ };  
class SingingWaiter : public Waiter, public Singer { /* ... */ };  
// 现在只有一个Worker对象
```

4.4 完整的多重继承示例

C++

```

// workermi.h
#ifndef WORKERMI_H
#define WORKERMI_H

#include <string>
#include <iostream>

class Worker {
private:
    std::string fullname;
    long id;

protected:
    virtual void Data() const;
    virtual void Get();

public:
    Worker() : fullname("no one"), id(0L) {}
    Worker(const std::string& s, long n) : fullname(s), id(n) {}
    virtual ~Worker() = 0; // 纯虚析构函数
    virtual void Set() = 0;
    virtual void Show() const = 0;
};

class Waiter : virtual public Worker {
private:
    int panache;

protected:
    void Data() const;
    void Get();

public:
    Waiter() : Worker(), panache(0) {}
    Waiter(const std::string& s, long n, int p = 0)
        : Worker(s, n), panache(p) {}
    Waiter(const Worker& wk, int p = 0)
        : Worker(wk), panache(p) {}

    void Set();
    void Show() const;
};

class Singer : virtual public Worker {
protected:
    enum {other, alto, contralto, soprano, bass, baritone, tenor};

```

```

enum {Vtypes = 7};

private:
    static char* pv[Vtypes];
    int voice;

protected:
    void Data() const;
    void Get();

public:
    Singer() : Worker(), voice(other) {}
    Singer(const std::string& s, long n, int v = other)
        : Worker(s, n), voice(v) {}
    Singer(const Worker& wk, int v = other)
        : Worker(wk), voice(v) {}

    void Set();
    void Show() const;
};

class SingingWaiter : public Singer, public Waiter {
protected:
    void Data() const;
    void Get();

public:
    SingingWaiter() {}
    SingingWaiter(const std::string& s, long n, int p = 0, int v = other)
        : Worker(s, n), Waiter(s, n, p), Singer(s, n, v) {}
    SingingWaiter(const Worker& wk, int p = 0, int v = other)
        : Worker(wk), Waiter(wk, p), Singer(wk, v) {}

    void Set();
    void Show() const;
};

#endif

```

4.5 虚基类的构造函数规则

使用虚基类时，构造函数必须显式调用虚基类的构造函数：

```
SingingWaiter(const Worker& wk, int p = 0, int v = other)
: Worker(wk),      // 必须显式调用虚基类构造函数
  Waiter(wk, p),
  Singer(wk, v) {}
```

4.6 解决方案：模块化方法

为避免重复显示信息，使用模块化方法：

CPP

```
void Worker::Data() const {
    cout << "Name: " << fullname << endl;
    cout << "Employee ID: " << id << endl;
}

void Waiter::Data() const {
    cout << "Panache rating: " << panache << endl;
}

void Singer::Data() const {
    cout << "Vocal range: " << pv[voice] << endl;
}

void SingingWaiter::Show() const {
    cout << "Category: singing waiter\n";
    Worker::Data(); // 只调用一次Worker数据
    Data();         // 调用组合的数据
}

void SingingWaiter::Data() const {
    Singer::Data();
    Waiter::Data();
}
```

5. 类模板

5.1 类模板基本概念

类模板允许我们创建通用的类定义，其中类型作为参数：

cpp

```

template <class Type> // 或 template <typename Type>
class Stack {
private:
    enum {MAX = 10};
    Type items[MAX];
    int top;

public:
    Stack();
    bool isempty();
    bool isfull();
    bool push(const Type& item);
    bool pop(Type& item);
};

```

5.2 模板类方法定义

```

cpp

template <class Type>
Stack<Type>::Stack() {
    top = 0;
}

template <class Type>
bool Stack<Type>::push(const Type& item) {
    if (top < MAX) {
        items[top++] = item;
        return true;
    } else {
        return false;
    }
}

template <class Type>
bool Stack<Type>::pop(Type& item) {
    if (top > 0) {
        item = items[--top];
        return true;
    } else {
        return false;
    }
}

```

5.3 使用模板类

cpp

```
#include "stacktp.h"
#include <string>

int main() {
    Stack<int> kernels;      // int类型的栈
    Stack<std::string> colonels; // string类型的栈

    kernels.push(3);
    kernels.push(35);

    std::string temp;
    colonels.push("Colonel Sanders");
    colonels.pop(temp);

    return 0;
}
```

5.4 非类型参数

模板可以包含非类型参数：

cpp

```
template <class T, int n>
class ArrayTP {
private:
    T ar[n];

public:
    ArrayTP() {}
    explicit ArrayTP(const T& v);
    virtual T& operator[](int i);
    virtual T operator[](int i) const;
};

// 使用
ArrayTP<double, 12> eggweights; // 12个double的数组
ArrayTP<int, 100> stuff;        // 100个int的数组
```

5.5 模板具体化

5.5.1 隐式实例化

cpp


```
ArrayTP<int, 100> stuff; // 编译器生成ArrayTP<int, 100>类
```

5.5.2 显式实例化

CPP

```
template class ArrayTP<string, 100>; // 强制实例化
```

5.5.3 显式具体化

CPP

```
template <>
class SortedArray<const char*> {
    // 专门为const char*定制的版本
    // 使用strcmp而不是>来比较
};
```

5.5.4 部分具体化

CPP

```
// 通用模板
template <class T1, class T2>
class Pair { /* ... */ };

// 部分具体化：T2固定为int
template <class T1>
class Pair<T1, int> { /* ... */ };

// 使用
Pair<double, double> p1; // 使用通用模板
Pair<double, int> p2;    // 使用部分具体化
```

5.6 模板参数

5.6.1 多个类型参数

CPP

```
template <class T1, class T2>
class Pair {
private:
    T1 a;
    T2 b;

public:
    Pair(const T1& aval, const T2& bval) : a(aval), b(bval) {}
    T1& first() { return a; }
    T2& second() { return b; }
    T1 first() const { return a; }
    T2 second() const { return b; }
};

// 使用
Pair<string, int> ratings("The Purpled Duck", 5);
```

5.6.2 默认模板参数

```
CPP

template <class T1, class T2 = int>
class Topo { /* ... */ };

Topo<double, double> m1; // T1=double, T2=double
Topo<double> m2;         // T1=double, T2=int
```

5.6.3 模板作为参数

CPP

```
template <template <typename T> class Thing>
class Crab {
private:
    Thing<int> s1;
    Thing<double> s2;

public:
    bool push(int a, double x) {
        return s1.push(a) && s2.push(x);
    }
    bool pop(int& a, double& x) {
        return s1.pop(a) && s2.pop(x);
    }
};

// 使用
Crab<Stack> nebula; // Stack必须是模板类
```

5.7 成员模板

CPP

```

template <typename T>
class beta {
private:
    template <typename V> // 嵌套模板类
    class hold {
    private:
        V val;
    public:
        hold(V v = 0) : val(v) {}
        void show() const { cout << val << endl; }
        V Value() const { return val; }
    };

    hold<T> q;
    hold<int> n;

public:
    beta(T t, int i) : q(t), n(i) {}

    template <typename U> // 模板方法
    U blab(U u, T t) {
        return (n.Value() + q.Value()) * u / t;
    }

    void Show() const { q.show(); n.show(); }
};

```

5.8 模板别名 (C++11)

CPP

```

template <typename T>
using arrtype = std::array<T, 12>;

arrtype<double> gallons; // 等价于std::array<double, 12>
arrtype<int> days;       // 等价于std::array<int, 12>
arrtype<std::string> months; // 等价于std::array<std::string, 12>

```

6. 实践练习

6.1 基础练习：包含vs继承

练习1：实现一个Computer类

CPP

// 要求: Computer类包含CPU和Memory对象

```
#include <string>
```

```
class CPU {  
private:  
    std::string brand;  
    double speed; // GHz  
  
public:  
    CPU(const std::string& b = "Intel", double s = 2.0)  
        : brand(b), speed(s) {}  
    void show() const;  
    std::string getBrand() const { return brand; }  
    double getSpeed() const { return speed; }  
};
```

```
class Memory {  
private:  
    int size; // GB  
  
public:  
    Memory(int s = 8) : size(s) {}  
    void show() const;  
    int getSize() const { return size; }  
};
```

// 使用包含实现Computer类

```
class Computer {  
private:  
    CPU processor;  
    Memory ram;  
    std::string model;  
  
public:  
    Computer(const std::string& m, const CPU& c, const Memory& mem)  
        : model(m), processor(c), ram(mem) {}  
  
    void showSpecs() const;  
    double getPerformanceScore() const;  
};
```

6.2 模板练习

练习2: 实现一个通用的Queue模板

```

template <class T>
class Queue {
private:
    enum {Q_SIZE = 10};
    T* items;
    int qsize;
    int front;
    int rear;
    int count;

public:
    Queue(int qs = Q_SIZE);
    ~Queue();
    Queue(const Queue& q);
    Queue& operator=(const Queue& q);

    bool isempty() const { return count == 0; }
    bool isfull() const { return count == qsize; }
    int queuecount() const { return count; }
    bool enqueue(const T& item);
    bool dequeue(T& item);
};

```

// 实现构造函数和方法

```

template <class T>
Queue<T>::Queue(int qs) : qsize(qs), front(0), rear(0), count(0) {
    items = new T[qsize];
}

```

```

template <class T>
Queue<T>::~~Queue() {
    delete[] items;
}

```

```

template <class T>
bool Queue<T>::enqueue(const T& item) {
    if (isfull())
        return false;
    items[rear] = item;
    rear = (rear + 1) % qsize;
    count++;
    return true;
}

```

```

template <class T>
bool Queue<T>::dequeue(T& item) {

```

```
if (isEmpty())  
    return false;  
item = items[front];  
front = (front + 1) % qsize;  
count--;  
return true;  
}
```

6.3 多重继承练习

练习3：实现Person、Worker、Student的多重继承

CPP

```
class Person {
protected:
    std::string firstName;
    std::string lastName;

public:
    Person(const std::string& fn = "", const std::string& ln = "")
        : firstName(fn), lastName(ln) {}
    virtual void show() const;
    virtual ~Person() {}
};
```

```
class Worker : virtual public Person {
protected:
    std::string company;
    double salary;

public:
    Worker(const std::string& fn = "", const std::string& ln = "",
           const std::string& c = "", double s = 0.0)
        : Person(fn, ln), company(c), salary(s) {}
    virtual void show() const;
};
```

```
class Student : virtual public Person {
protected:
    std::string school;
    double gpa;

public:
    Student(const std::string& fn = "", const std::string& ln = "",
            const std::string& sch = "", double g = 0.0)
        : Person(fn, ln), school(sch), gpa(g) {}
    virtual void show() const;
};
```

```
class WorkingStudent : public Worker, public Student {
public:
    WorkingStudent(const std::string& fn, const std::string& ln,
                   const std::string& c, double s,
                   const std::string& sch, double g)
        : Person(fn, ln), Worker(fn, ln, c, s), Student(fn, ln, sch, g) {}
    void show() const;
};
```


7. 总结

7.1 代码重用方式对比

方式	关系类型	优点	缺点	适用场景
公有继承	is-a	多态性，接口继承	紧耦合	真正的is-a关系
包含	has-a	简单，灵活，松耦合	需要包装器方法	大多数has-a关系
私有继承	has-a	访问保护成员，可重定义虚函数	复杂，紧耦合	需要访问内部实现
多重继承	多种关系	功能强大	复杂，易产生问题	确实需要多个基类
模板	通用	类型安全，高效	编译时确定，代码膨胀	通用容器和算法

7.2 最佳实践

1. **优先使用包含**：大多数has-a关系用包含实现
2. **谨慎使用多重继承**：避免复杂的继承层次
3. **合理使用模板**：为通用性和类型安全
4. **明确关系类型**：is-a用公有继承，has-a用包含
5. **使用虚基类**：解决多重继承中的二义性

7.3 常见错误避免

1. **不要混淆is-a和has-a关系**
2. **注意模板的编译时特性**
3. **小心多重继承的构造函数调用顺序**
4. **正确使用虚基类解决二义性**
5. **模板定义和声明放在同一个头文件中**

7.4 进阶学习建议

1. 深入学习STL（标准模板库）
2. 了解现代C++的智能指针
3. 学习C++11/14/17的新特性
4. 掌握设计模式在C++中的应用
5. 理解内存管理和RAII原则

附录：快速参考

A.1 语法速查

```

// 包含
class Container {
    OtherClass member; // 包含其他类的对象
};

// 私有继承
class Derived : private Base {
    // 使用Base::method; // 重新公开某个方法
};

// 多重继承
class Child : public Parent1, public Parent2 {
    // 构造函数必须初始化所有基类
};

// 虚基类
class Child : virtual public Base {
    // 避免多重继承中的重复基类
};

// 类模板
template <typename T>
class Container {
    T data;
public:
    void set(const T& value) { data = value; }
};

// 非类型参数
template <typename T, int Size>
class Array {
    T arr[Size];
};

// 模板特化
template <>
class Container<char*> {
    // 专门为char*的实现
};

```

A.2 关键概念记忆

代码重用的黄金法则：

- **is-a关系** → 公有继承
- **has-a关系** → 包含（首选）或私有继承

- 通用代码 → 模板

多重继承记忆口诀：

- 虚基类解决重复问题
- 作用域解析消除二义性
- 构造函数必须显式调用虚基类

模板使用要点：

- 声明和定义放在同一文件
 - 显式实例化用template class
 - 特化用模板<>
-

如何生成PDF

由于我无法直接生成PDF，请使用以下方法之一：

方法一：使用在线转换器

1. 复制上面的Markdown内容
2. 访问在线Markdown转PDF网站，如：
 - <https://www.markdowntopdf.com/>
 - <https://dillinger.io/>
 - <https://pandoc.org/try/>
3. 粘贴内容并下载PDF

方法二：使用编辑器

1. 复制内容到支持Markdown的编辑器：
 - Typora（推荐）
 - 标记文本
 - 黑曜石
 - VS Code with Markdown 预览增强功能
2. 使用编辑器的导出PDF功能

方法三：使用命令行工具

如果您有Node.js环境：

抨击

```
npm install -g markdown-pdf  
echo "内容" > handbook.md  
markdown-pdf handbook.md
```

方法四：使用Pandoc

如果安装了Pandoc：

抨击

```
pandoc handbook.md -o handbook.pdf
```

方法五：使用浏览器

1. 将内容保存为.md文件
2. 在支持Markdown的在线编辑器中打开
3. 使用浏览器的打印功能，选择“保存为PDF”

推荐设置（PDF格式优化）

为了获得最佳的PDF效果，建议：

- 字体：宋体或Times New Roman
- 字号：12pt正文，14pt标题
- 页边距：2.5cm
- 行距：1.5倍
- 添加页码和目录

这个自学手册涵盖了C++代码重用的所有核心概念，包含完整的代码示例和实践练习。建议按章节顺序学习，每章都要通过编写和运行代码来验证理解程度。